

P E E P A L

Onboarding Guide

A complete reference for institute users, administrators, and the engineering team.

Product	Peepal ERP (rosERP)
Audience	End users, institute admins, developers, ops
Document version	1.0
Date	August 2026
Stack	Next.js 14 · TypeScript · Redux Toolkit · Apollo · Go Fiber · gqlgen · GORM · PostgreSQL

Contents

PART 1 · USING PEEPAL

- 1** What Peepal is
- 2** Getting access and signing in
- 3** Finding your way around
- 4** Roles and what each one sees
- 5** Administrator: first-week checklist
- 6** Module reference
- 7** Everyday workflows
- 8** Troubleshooting for users

PART 2 · TECHNICAL AND OPERATIONS

- 9** Architecture overview
- 10** System requirements
- 11** Local installation
- 12** Codebase conventions
- 13** Building a feature end to end
- 14** Release and deployment
- 15** Cloud infrastructure and environment
- 16** Security, backups and support
- 17** Quick reference

APPENDIX

- A** Glossary
- B** Industry availability matrix

PART ONE

Using Peepal

For students, teachers, staff, clinicians and institute administrators. No technical background needed.

1. What Peepal is

Peepal is a multi-tenant ERP that runs the daily operations of an institution: the people, the academic or clinical calendar, money, facilities, approvals and reporting, all in one place.

Three ideas explain almost everything about how the product behaves.

1.1 Every institution is a tenant

Your institution is a **tenant** with its own isolated data. You reach it either at a subdomain such as `https://yourschool.roserp.workvar.com` or through a path such as `/yourschool/dashboard`. Nobody outside your tenant can see your records, and every row of business data is filtered by tenant on the server.

1.2 The product adapts to your industry

Peepal supports several verticals. The industry chosen for your tenant decides which modules exist at all, and the vocabulary used across the interface.

Industry	What the sidebar emphasises
Education	Students, courses, subjects, marks, exams, timetable, fees, hostel, transport, library
Healthcare	Patients, appointments, OPD visits, admissions, laboratory, radiology, nursing, pharmacy, billing and claims
Corporate	Employees, payroll, leaves, approvals, finance, procurement, learning
Nonprofit	Shared people, approvals, finance and reporting modules

Shared modules such as Employees, Leaves, Payroll, Approvals, Finance, Announcements and Reports appear for every industry. A hospital never sees Marks; a college never sees Radiology. This is enforced on the server, not just hidden in the menu.

Terminology layer. Words in the interface are configurable per tenant. A "Student" may read as "Trainee", a "Course" as "Programme", a "Department" as "Unit". If the labels in this guide differ slightly from your screen, the underlying feature is the same.

1.3 What you can see is decided in two layers

1. **Subscription** decides which module families your institution has paid for.
2. **Access matrix** decides, per role, which modules within that set you may view, create, edit or delete.

If a colleague sees a menu item you do not, the difference is almost always one of those two layers, not a bug.

2. Getting access and signing in

2.1 Accounts are created for you

Peepal has no public sign-up. Your institute administrator creates your account, either directly with a password, or by sending you an **invitation email** containing a one-time setup link where you choose your own password.

Invitation links expire. If your setup link no longer works, ask your administrator to resend the invite rather than trying to register yourself.

2.2 Email is not always your username

Some institutions run without staff or student email addresses. In that case you sign in with your **Employee ID** or **Roll Number** instead, and only administrators hold an email-based account. Your administrator will tell you which mode your tenant uses.

2.3 Signing in

1. Open your tenant URL, for example <https://yourschool.roserp.workvar.com/login>.
2. Enter your email (or ID) and password.
3. You land on the Dashboard scoped to your role.

Your session is held in a secure, browser-managed cookie that lasts up to **24 hours**. After that you are asked to sign in again. First-party cookies must be allowed for the Peepal domain, otherwise login will appear to succeed and then bounce you back.

2.4 First things to do

- Change the password you were given, from **My Profile**.
- Check that your name, department and contact details are correct, and report corrections to your administrator.
- Open **Notifications** so you know where alerts arrive.

3. Finding your way around

3.1 The layout

Area	What it does
Left sidebar	All modules you are allowed to open, grouped into collapsible sections such as Attendance, Academic, Fees, Facilities and Organisation. On a small screen it becomes a slide-out menu.
Top bar	Global search, the notification bell, and your user menu.
Search	Jumps straight to a page or record without hunting through the menu. Fastest way to move around.
Notification bell	Approvals waiting on you, announcements, and system alerts.
User menu	My Profile, theme, and sign out.
Quota banner	A persistent strip that appears when your institution is at or over its licensed student or employee count. Only an administrator can resolve it.

3.2 Sections you will see in the sidebar

The sidebar only renders sections that contain at least one module you can open, so your menu will be shorter than the full list below.

Section	Contains
Top level	Dashboard, Ask PeepalAI, My Portal, My Fees, My Hall Tickets, Admins, Employees, Students, My Approvals, Org Structure
Attendance	Attendance, Attendance Summary, Shortage List, Attendance Export
Academic	Marks, Results, Courses, Subjects, Curriculum, Exams, Exam Schedules, Timetable
Exam Cell	Question Bank, Question Papers, Hall Tickets
Patient Care	Patients, Appointments, OPD Visits, Admissions, Schedules
Diagnostics	Laboratory, Radiology
Ward & Theatre	Nursing, OT Schedule, Triage
Support Services	Blood Bank, Ambulance, Dietary, Telemedicine, Referrals
Billing & Claims	Billing, Claims
Inventory / Procurement	Pharmacy, Stores, Vendors, Purchase Orders
Leaves	Leaves, Leave Types
Payroll	Payroll, Salary Templates, Salary Assignments
Fees	Categories, Add-ons, Structures, Allocations, Payments, Student Fees, Overview
Finance	Chart of Accounts, General Ledger, Receivables, Payables
Reports	Attendance, Marks, Fees, Payroll, Leaves
Facilities	Hostel, Transport, Library, Wards & Beds
Campus Life	Events

Section	Contains
Learning	Goal Library, Assignments, My Learning
Communication	Announcements, Notifications
Organisation	Org Profile, Departments, Academic Years, Roles, Access Control, Audit Trail, Email, Holidays, Calendar, Approval Types, Approval Flows

4. Roles and what each one sees

Peepal ships six built-in roles. Institutions layer custom roles on top of these, so your job title may be "Receptionist" or "Pharmacist" while the system still treats you as staff underneath.

Role	Typical scope
Admin	Full control of the tenant. Creates people and accounts, configures roles, access, approvals, fees, payroll, academic years and holidays. Sees every report and the audit trail.
Teacher	Teaching and clinical-practitioner work: attendance, marks, results, courses, subjects, exams, timetable, assignments, leaves, approvals assigned to them. In healthcare tenants this is the clinician role, with My Schedule.
Staff	Operational work: employees, attendance, fee payments, facilities, inventory and procurement, billing and claims, front desk, leaves.
Student	Self-service only: My Portal, My Fees, My Grades, My Hall Tickets, My Assignments, My Mess, plus attendance, marks, timetable, library and events in read-only form.
Patient	Healthcare self-service: My Health, plus dashboard, notifications and profile. Nothing else.
Super admin	Platform operator, outside any single tenant. Manages tenants, plans, subscriptions, module configuration and platform analytics from the <code>/super</code> console.

4.1 Custom roles

Administrators create named roles under **Organisation → Roles** and grant them permissions per module under **Access Control**. Healthcare tenants are seeded with Receptionist, Doctor and Pharmacist roles automatically. A custom role can never exceed the ceiling of the base role it sits on, so a custom role built on staff cannot reach admin-only pages.

4.2 Role-locked pages

A handful of pages hold data that only exists for one role, and are therefore hidden from everyone else including administrators: My Portal, My Fees, My Grades, My Hall Tickets, My Assignments, My Mess, My Health and My Schedule. If you are an administrator wanting to see a student's fees, use **Fees → Student Fees** rather than the student portal.

5. Administrator: first-week checklist

Work through this in order. Each step depends on the ones above it.

Day 1 · Secure the tenant

1. Receive your tenant URL and administrator credentials from the platform owner.
2. Sign in and change the password immediately.
3. Open **Organisation → Org Profile** and set the institution name, address, logo and identity policy (whether staff and students need email addresses).
4. Confirm your subscription and enabled modules with support, so you are not configuring something you cannot use.

Day 2 · Structure

1. **Departments:** create the organisational units everything else hangs off.
2. **Academic Years** (education): define the current year and term.
3. **Holidays** and **Calendar:** load the annual calendar so attendance and leave calculations are correct from the start.
4. **Roles** and **Access Control:** create the roles your institution actually uses and grant least-privilege permissions.

Day 3 · People

1. **Employees:** add staff and teachers. Creating an employee also creates their login account in the same step, so choose the role and either set a password or send an invite.
2. **Students:** enrol students the same way, or use bulk upload for a whole intake.
3. **Admins:** add any additional administrators, sparingly.

Day 4 · Academic or clinical setup

1. Education: Courses, Subjects, Curriculum, Exams and exam types, Grading scheme, Timetable.
2. Healthcare: Schedules for clinicians, Wards and Beds, Laboratory test catalogue, Pharmacy drug list.

Day 5 · Money and process

1. **Fees:** Categories, then Structures per course, then Allocations, which generate each student's fee record and instalments.
2. **Payroll:** Salary Templates, then Salary Assignments per employee.
3. **Leave Types** and **Approval Types / Approval Flows:** define who approves what, in what order.
4. **Email:** configure SMTP so invitations and notifications can actually be delivered.

Day 5 · Verify

1. Sign in as one user per major role and confirm each sees only what they should.
2. Submit one test leave request and approve it, to prove the approval chain works.
3. Record one fee payment and download the receipt.
4. Open **Audit Trail** and confirm your test actions were logged.

Order matters. Fees cannot be allocated before structures exist, structures need courses, courses need departments. Working top-down saves rework.

6. Module reference

What each area of the product is for, and who normally uses it. Availability depends on your industry and subscription.

6.1 People and organisation

Module	Purpose	Typical users
Dashboard	Role-aware landing page with live charts and counts sourced from the reporting layer.	Everyone
Employees	Staff and teacher records, departments, reporting lines. Creating a record creates the login account with it.	Admin, staff
Students	Enrolment, course and batch assignment, roll numbers, contact details.	Admin, teacher
Admins	The short list of accounts holding administrator rights.	Admin
Org Structure	Visual reporting hierarchy across the institution.	Admin, teacher, staff
Organisation	Profile, departments, academic years, roles, access control, holidays, calendar, email, approvals, audit trail.	Admin
Audit Trail	Who changed what and when, including logins and bulk uploads (flagged separately).	Admin

6.2 Academic (education)

Module	Purpose	Typical users
Attendance	Mark attendance individually or in bulk, for students and employees. Summary, shortage list and export views sit alongside it.	Admin, teacher, staff
Marks	Enter assessment marks. The assessment type list and maximum marks are driven by the Exams module.	Admin, teacher
Results	Consolidated published results per student.	Admin, teacher, student
Courses / Subjects	Programmes offered, and the subjects within them, including course outcomes and unit plans.	Admin, teacher
Curriculum	Assign subjects to each course and semester.	Admin
Exams / Exam Schedules	Assessment definitions (department-scoped) and the dated exam calendar.	Admin, teacher
Grading	Per-tenant grading scheme and grade bands. Drives live SGPA and CGPA from published marks.	Admin
Timetable	Period-wise schedule per course, semester and section.	Everyone
Exam Cell	Question Bank, generated Question Papers, and QR-coded Hall Tickets.	Admin, teacher, staff
Assignments	Set work for a cohort; students submit and track it from their portal.	Admin, teacher

6.3 Clinical (healthcare)

Module	Purpose	Typical users
Patients	Patient master with UHID, demographics and history.	Admin, clinician, staff
Appointments	Booking against clinician schedules, which gate available slots.	Front desk, clinician
OPD Visits	Encounters with CR numbers, notes and orders.	Clinician
Admissions / Wards & Beds	Inpatient admission, transfer and discharge, with a live ward and bed visualiser and discharge summary PDF.	Admin, staff
Laboratory / Radiology	Test catalogues, orders, results and reports.	Clinician, staff
Nursing	Medication administration record and vitals.	Nursing staff
OT Schedule / Triage	Theatre scheduling and emergency triage.	Clinician, staff
Pharmacy	Drug stock with atomic dispensing against prescriptions.	Pharmacist
Billing / Claims	Billable services, invoices, receipts and payments; insurance and TPA claims with an approval engine.	Admin, billing staff
Support services	Blood bank, ambulance, dietary, telemedicine and referrals with commission settlement.	Staff, clinician
My Health / My Schedule	Patient self-service portal, and the clinician's own calendar.	Patient / clinician

6.4 Money

Module	Purpose	Typical users
Fees	Categories define what is charged; Structures set course-scoped amounts per year; Allocations turn those into per-student fees and instalments by frequency; Payments record receipts; Overview and Student Fees show outstanding dues.	Admin, accounts staff
Payroll	Salary templates, per-employee assignments, and payslip generation as PDF.	Admin
Finance	Chart of accounts, general ledger, receivables and payables.	Admin
Procurement	Vendors, purchase orders and stores, feeding inventory and pharmacy.	Admin, staff

6.5 Facilities, campus life and communication

Module	Purpose	Typical users
Hostel	Bed-wise allocation through a visual block and room map, with gender-restricted blocks. Mess menus and meal attendance sit here.	Admin, staff
Transport	Routes, vehicles, student allocations, plus live vehicle tracking and driver attendance.	Admin, staff
Library	Catalogue, issue and return.	Everyone

Module	Purpose	Typical users
Events	Institution calendar with custom, tenant-defined categories.	Everyone
Learning	Goal library, goal assignments, and each person's own learning track.	Admin, everyone
Announcements / Notifications	Broadcast messages and personal alerts.	Everyone
Leaves	Apply, track and approve leave against configurable leave types scoped by role.	Everyone except students
Duty Roster	Shift rostering across departments, available in every industry.	Admin, staff
Ask PeepalAI	Ask questions about your own data in plain language. Answers are scoped to what your role is allowed to see.	Everyone

7. Everyday workflows

7.1 Bulk upload with CSV

Most list pages that accept large volumes carry a **Bulk Upload** button. The flow is the same everywhere.

1. Click **Bulk Upload** and download the field guide, which lists every column, whether it is required, and the accepted values.
2. Fill the CSV. Keep referenced values exactly as they appear in the application: department names, course codes, roll numbers and similar.
3. Upload and review the validation result before committing.
4. Rows referencing something that does not exist fail individually; the rest still import. Fix and re-upload only the failures.

Reference values are validated, not created. A misspelled department will fail the row rather than silently creating a new department. Create the reference data first.

7.2 Applying for and approving leave

1. Open **Leaves** and submit a request against an available leave type.
2. The request routes through the approval flow configured for that type.
3. Approvers see it under **My Approvals** and in the notification bell.
4. Each decision is recorded in the audit trail.

7.3 Recording a fee payment

1. Go to **Fees → Payments** and search for the student.
2. Select the outstanding instalment and record the amount, mode and reference.
3. Download the receipt PDF, which is generated on the server.
4. Confirm the balance updated in **Fees → Overview**.

7.4 Marking attendance

1. Open **Attendance**, choose the date, course, semester and section.
2. Mark the roster individually, or use bulk mark for a whole class.
3. Use **Attendance Summary** for percentages and **Shortage List** to find students below the threshold.
4. **Attendance Export** produces a downloadable file for external reporting.

7.5 Documents you can download

Document	Where it comes from
Payslip	Payroll
Fee receipt and fee schedule	Fees → Payments / Student Fees
Grade report	Grading, and My Grades for students
Hall ticket	Exam Cell, and My Hall Tickets for students
Invoice and receipt	Billing (healthcare)
Discharge summary	Admissions (healthcare)

Document	Where it comes from
Institution brochure	Public brochure page, editable by administrators

8. Troubleshooting for users

Symptom	Most likely cause and fix
Login succeeds then returns to the login page	Cookies are blocked for the Peepal domain, or a captive-portal network is stripping them. Allow first-party cookies and retry outside the portal.
Signed out unexpectedly	Sessions last up to 24 hours. Sign in again; nothing is lost apart from an unsaved form.
A menu item a colleague has is missing for you	Your role lacks the permission, or the page is role-locked. Ask your administrator to check Access Control.
A whole section is missing for everyone	The module is not part of your subscription, or does not exist for your industry. Contact support.
Nobody can sign in at all	The subscription may have lapsed. Login is blocked without an active subscription. Contact your platform owner.
A banner says you are over quota	Your licensed student or employee count is exceeded. New records are blocked until an administrator upgrades the plan or removes records.
Bulk upload rejects rows	A referenced department, course or roll number does not match an existing record. Create the reference first, then re-upload only the failed rows.
Invitation email never arrived	Check spam, then ask your administrator to confirm SMTP is configured under Organisation → Email and resend.
A page renders but tables are cramped	Dense admin tables and bulk tools are designed for desktop. Use a screen of at least 1280×720.

Reporting a problem. Include the time, your tenant name, your role, the page you were on, and what you expected to happen. That is usually enough for support to reproduce it without a follow-up round trip.

Escalation path: institute administrator, then Peepal support, then engineering on call.

PART TWO

Technical and operations

For developers joining the codebase, and for the team that installs, deploys and operates Peepal.

9. Architecture overview

```

Browser (tenant slug, or /super)
  |
  v
Next.js 14 frontend --PM2--> :3000
- App Router, TypeScript, Tailwind
- Apollo Client -> GraphQL
- axios -> thin REST (auth, uploads, bulk CSV, super-admin)
- Redux Toolkit -> auth, org chrome, terminology
- JWT in httpOnly cookie `peepal_token`
  | rewrite /api/* -> NEXT_PUBLIC_API_URL
  v
Go Fiber API --PM2--> :8080
- GraphQL (gqlgen) at POST /api/v1/graphql <- default surface
- REST for login, files, bulk CSV, super-admin
- GORM (pgx), JWT + bcrypt, tenant scoping, field middleware
  |
  v
PostgreSQL (Aiven-managed)
- One shared database; every business row carries tenant_id
- Schema changes only via `go run main.go --migrate`

```

9.1 Repository layout

```

Peepal/
  backend/   Go Fiber API (GraphQL + thin REST)
  frontend/  Next.js 14 app
  docs/      identity modes, multi-industry, compliance notes
  installer/ native on-prem installers and the peepal-agent supervisor

```

9.2 Backend anatomy

Path	Contents
<code>main.go</code>	Entry point. Reads <code>--migrate</code> and other flags, wires config, DB, middleware and routes.
<code>graph/</code>	gqlgen schema and resolvers, split per module as <code><module>.resolvers.go</code> . This is where new work goes.
<code>models/</code>	GORM models. Every business model carries <code>tenant_id</code> and a UUID hook.
<code>handlers/</code>	Remaining REST handlers: auth, terminology, super-admin, file streaming.
<code>handlers/bulk/</code>	Shared bulk-CSV framework: a registry plus one <code>schema_*.go</code> per resource, self-registered via <code>init()</code> .
<code>middleware/</code>	JWT auth, role guards, tenant scoping, access field middleware, audit.
<code>audit/</code>	Audit recording. GraphQL mutations are logged automatically; REST handlers must call <code>Record</code> explicitly.
<code>pdf-template/</code>	Server-side PDF generation for payslips, receipts, fee schedules and grade reports.
<code>mailer/</code> , <code>invites/</code>	SMTP delivery and the invitation flow.

9.3 Frontend anatomy

Path	Contents
<code>app/</code>	App Router. <code>app/[tenant]/(dashboard)/...</code> for tenant pages, <code>app/(super-admin)/super/...</code> for the platform console. Page files stay thin.
<code>components/</code>	All real UI. Page logic lives under <code>components/pages/...</code> and is imported by the route file.
<code>store/</code>	Redux Toolkit slices: auth, access, terminology, org chrome. Shared hooks in <code>store/hooks.ts</code> .
<code>lib/</code>	<code>access.ts</code> (industry map, role-locked modules, path-to-module resolution), Apollo client, helpers.
<code>constants/navigation/</code>	<code>BASE_MODULES</code> and the sidebar <code>navConfig.ts</code> . A module must be registered here to appear.
<code>types/</code>	Shared TypeScript types, including <code>Role</code> and <code>TenantType</code> .
<code>tests/</code>	Playwright end-to-end suite, ordered with LIFO cleanup.

9.4 Multi-tenancy and access, in three layers

1. **Industry.** `models/access_industries.go` on the server and `MODULE_INDUSTRIES` in `lib/access.ts` decide whether a module exists for the tenant at all.
2. **Subscription.** `models/subscription_modules.go` maps fine-grained modules onto coarse subscription modules; effective modules are enforced through `myAccess` and the access field middleware. Super admin is exempt; administrators are not.
3. **Access matrix.** Per-tenant role to module CRUD grants (`AccessRule`), edited at `/org/access-control`, with a role ceiling derived from resolver `requireRole` guards so navigation never links to a forbidden page.

Golden rule. Never query a business table without a tenant filter.

10. System requirements

10.1 Developer workstation

Resource	Minimum	Recommended
OS	macOS 12+, Windows 10/11 with WSL2, or Ubuntu 22.04+	macOS 14+ or Ubuntu 24.04 LTS
CPU / RAM	2 cores, 8 GB	4+ cores, 16 GB
Disk	10 GB free	25 GB free SSD
Node.js	18.x LTS	20.x LTS
Package manager	pnpm 8+	pnpm 9+
Go	1.21+	1.22+ (project targets 1.25)
Git	2.40+	Latest
Network	Access to GitHub and the managed Postgres	Same, plus VPN if org policy requires

Optional: Docker for isolated backend builds, PM2 to mirror production process management locally.

10.2 Production or staging host

Assumes the Next.js app and the Go API on one host, with the database managed externally.

Resource	Minimum	Recommended
OS	Ubuntu 22.04 LTS	Ubuntu 24.04 LTS
CPU / RAM	2 vCPU, 4 GB	4 vCPU, 8 GB (16 GB for heavy reporting)
Disk	40 GB SSD	80+ GB SSD, separate volume for <code>backend/uploads/</code>
Process manager	PM2	PM2 with <code>pm2 startup</code> and <code>pm2 save</code>
Reverse proxy	Nginx or Caddy with TLS	Nginx with HTTP/2, compression and HSTS
Outbound	HTTPS to the managed Postgres; DNS for the tenant wildcard	Same, plus monitoring egress

10.3 Managed PostgreSQL

Resource	Minimum	Recommended
PostgreSQL	14+	15 or 16
Storage	10 GB	50+ GB with growth alerts
Availability	Single node (non-production)	Multi-node HA
Backups	Daily snapshots	Daily plus point-in-time recovery
Connectivity	TLS required, IP allowlist	Private networking or VPC peering

10.4 End-user devices

Chrome, Edge, Firefox or Safari within the last two major versions; 1280×720 minimum, 1440×900 recommended; first-party cookies allowed for the Peepal domain. Mobile browsers work for most modules, but dense admin tables and bulk tools are desktop-first.

11. Local installation

11.1 Prerequisites

1. Install Node.js 18+ and pnpm.
2. Install Go 1.21+.
3. Get access to the backend and frontend remotes, and a non-production Postgres DSN.
4. Ask a platform owner for non-production credentials. Never use production admin passwords locally.

11.2 Backend

```
cd backend
cp .env.example .env
# set at minimum:
#   DB_PATH=<postgres DSN>
#   PORT=3001             (default in .env.example; config falls back to 8080)
#   COOKIE_DOMAIN=        (MUST be empty for localhost)

go mod tidy

# first boot, with schema sync
go run main.go --migrate

# subsequent boots, no schema changes
go run main.go
```

On first run a default administrator is seeded:

Field	Value
Email	admin@college.edu
Password	Admin@123

Change the seeded credentials before the instance is reachable by anyone else.

Migrations are opt-in. A normal restart does not run GORM AutoMigrate. Pass `--migrate` only when models changed. Other one-off maintenance flags exist, for example `--backfill-uhid`; check `main.go` before inventing a new script.

Never run `git checkout` or `git restore` in this repository. The working tree is significantly ahead of HEAD, and a checkout will destroy uncommitted work including the generated GraphQL schema.

11.3 Frontend

```
cd frontend
cp .env.local.example .env.local
# NEXT_PUBLIC_API_URL=http://localhost:3001 (match the backend PORT)
# NEXT_PUBLIC_GA_MEASUREMENT_ID=           (leave blank locally)
# NEXT_PUBLIC_CLARITY_PROJECT_ID=          (leave blank locally)

pnpm install
pnpm dev      # http://localhost:3000
```

`next.config` rewrites `/api/:path*` to `${NEXT_PUBLIC_API_URL}/api/:path*`. GraphQL requests go to `{API}/api/v1/graphql` with credentials included, and carry `X-Tenant-ID` when a tenant context is set.

11.4 Verification

Check	Expected
Backend starts	Listening on the configured port with no migration errors
<code>http://localhost:3000</code>	Marketing or login page renders
Sign in as a test admin	<code>peepal_token</code> cookie set, redirect into the dashboard
<code>/super/login</code>	Platform console reachable for authorised users
<code>/slug/login</code>	Tenant-scoped session

```
# end-to-end suite, with backend and pnpm dev running
pnpm test:install
cp tests/.env.example tests/.env
pnpm test
```

11.5 Optional: Docker for the backend

```
cd backend
docker build -t peepal-backend .
docker run -p 8080:8080 \
  -e DB_PATH="<dsn>" \
  -v peepal-uploads:/app/uploads \
  peepal-backend
```

12. Codebase conventions

12.1 GraphQL first

The backend exposes GraphQL and a shrinking set of REST routes side by side. **Default all new work to GraphQL.**

- Add features as gqlgen resolvers in a per-module `graph/<module>.resolvers.go`.
- Do not add REST routes or handlers unless GraphQL genuinely cannot serve the case: file streaming, third-party webhooks, CSV upload.
- Existing REST endpoints (auth and login, terminology, super-admin, bulk upload) stay as they are.
- On the frontend, call GraphQL queries and mutations rather than REST through axios.
- When extending an existing feature, match the API it already uses. Default any net-new surface to GraphQL.

12.2 Small files, split logic

Keep route files thin and put the real work in `components/pages/...`. Split large modules into several files rather than growing one. The same applies on the backend: one resolvers file per module, one bulk schema file per resource.

12.3 Naming across the boundary

gqlgen emits camelCase. A few legacy TypeScript types are snake_case, and nothing transforms between them automatically. A mismatch produces silent `undefined` reads on read and an HTTP 422 on write. Map explicitly at the slice boundary with `fromGql` and `toGqlInput` helpers.

12.4 Authentication

- The JWT lives in an httpOnly cookie named `peepal_token`, valid for 24 hours. Never store tokens in localStorage.
- Redux holds only a `hasSession` marker. That marker can drift from the real cookie, so always await logout before navigating, and call `useHydrateSession()` on login pages.
- Resolvers enforce authorisation with `requireAuth` and `requireRole`. Frontend guards are for user experience only.
- `COOKIE_DOMAIN` must be empty on localhost. A domain value is rejected by the browser and freezes a stale auth cookie, which shows up as `me()` returning the previous user.

12.5 Data integrity

- There are no database-level foreign keys. Bulk CSV rows validate their references through `handlers/bulk/reference_lookups.go`; a non-blank miss fails that row only.
- Deleting a person cascades every relation through `graph/cascade_delete.go` and `deleteUserCascade`. Extend those helpers whenever you add a per-person table.
- Audit logging is automatic for GraphQL mutations. REST handlers, including bulk row submission, must call the `audit` package's `Record` themselves.

12.6 Vocabulary

Shared UI must read its words from the terminology tokens and `roleLabels.ts`, never hardcoded education nouns. Role identifiers in the database never change; only the display label does.

13. Building a feature end to end

13.1 First day

1. Read this guide, the root `CLAUDE.md`, and `frontend/ONBOARDING.md`.
2. Install the toolchain and bring up both services.
3. Sign in and browse the in-product technical docs at `/tenant/docs/dev` and `/super/tech-docs`.
4. Open `/super/db-visualizer` to see the schema as an interactive map.
5. Run a focused Playwright smoke test if credentials allow.

13.2 The workflow

1. **Model.** Add a GORM model under `backend/models` with `tenant_id` and the UUID hook.
2. **Migrate.** `go run main.go --migrate`, once.
3. **API.** Write a gqlgen resolver in `graph/<module>.resolvers.go`. Guard it with `requireAuth` and, where relevant, `requireRole`.
4. **Access.** Register the new module id so it appears in the access matrix, and add it to `MODULE_INDUSTRIES` if it is industry-specific, or `ROLE_LOCKED_MODULES` if it is single-role self-service.
5. **Frontend.** Thin `app/.../page.tsx`, real logic in `components/pages/...`.
6. **State.** Apollo for server data, Redux for auth, chrome and terminology.
7. **Navigation.** Add the entry to `BASE_MODULES` and `navConfig.ts` so the tile appears.
8. **Bulk.** If the resource needs CSV import, add a `schema_*.go` under `handlers/bulk/` and drop `<BulkUploadButton resource="..." />` on the page. For resources whose creation has side effects, prefer a per-row GraphQL mutation loop instead.
9. **Tests.** Extend the Playwright hierarchy carefully; it is ordered with LIFO cleanup.

Forgetting step 4 is the classic mistake. A module missing from `ROLE_LOCKED_MODULES` leaks into the admin sidebar (administrators hold a blanket view grant) and then 404s when opened.

13.3 Regenerating GraphQL

Regenerate the schema from live introspection rather than hand-editing generated artefacts. After a migration, regenerate the frontend schema documentation with `pnpm gen:schema`. Note that a small number of mutations have been hand-wired into the generated code, so run a full regeneration deliberately and review the diff.

14. Release and deployment

14.1 Cadence

Channel	Frequency	Contents and impact
Hotfix	As needed, same day	Security issues, auth bugs, data-exposure fixes. Brief window if a restart is required.
Patch	Weekly	Bug fixes, copy, small UX. Rolling restart.
Minor	Every two to four weeks	New modules, new GraphQL fields, non-breaking schema. Run <code>--migrate</code> if models changed.
Major	Quarterly, planned	Breaking API or schema, role-model changes, infrastructure moves. Announced in advance with an admin checklist.

14.2 Weekly rhythm

Day	Activity
Monday	Triage; freeze candidates for the Tuesday patch.
Tuesday	Deploy to staging, smoke test, then production in a low-traffic window.
Wednesday to Thursday	Feature development; schema work on feature branches.
Friday	No production schema migrations unless hotfixing. Tag the weekly build.

14.3 Production install and deploy

```
# --- backend ---
cd backend
go build -o collegeerp .
pm2 start ecosystem.config.js

# apply schema only as a planned step of the release
./collegeerp --migrate

# --- frontend ---
cd ../frontend
pnpm install --frozen-lockfile
pnpm build
pm2 start ecosystem.config.js    # peepal-frontend -> next start :3000

pm2 save
pm2 startup
```

14.4 Release checklist

1. **Announce** the window to administrators if downtime or a migration is expected.
2. **Back up:** take a snapshot, or confirm the latest automated backup is green.
3. Deploy the **backend** first, kept compatible with the old frontend where possible.
4. If models changed, run `--migrate` **once** against the production DSN from a controlled shell.
5. Deploy the **frontend** and reload PM2.
6. **Smoke test:** super-admin login, one tenant login, one GraphQL list query, one file upload, one PDF download.

7. **Monitor** error logs and analytics for 30 to 60 minutes.
8. **Rollback** by restoring the previous PM2 binary and build. Restore the database only if the migration was destructive; prefer forward fixes.

14.5 Schema discipline

- Use expand, migrate, deploy, contract for breaking changes.
- Never run destructive SQL in production without a backup and a written rollback plan.
- Mount `backend/uploads/` on durable storage. Photos and generated PDFs live there, outside the binary, and must be in the backup policy.

14.6 Dependency updates

Class	Schedule
Security patches (Node, Go, Next, Fiber)	Within 7 days of the advisory
Routine npm and Go module bumps	Monthly on staging, then production
Major framework upgrades	Dedicated major release with a full regression run

15. Cloud infrastructure and environment

15.1 Topology

Component	Notes
Web app	Next.js under PM2 as <code>peepal-frontend</code> , port 3000, behind a reverse proxy.
API	Go Fiber binary under PM2, port 8080 in production, public hostname <code>api-roserp.workvar.com</code> .
Database	Aiven-managed PostgreSQL over TLS, DSN in <code>DB_PATH</code> , GORM with the <code>pgx</code> driver.
Files	<code>backend/uploads/</code> on a persistent disk: photos, payslips and generated PDFs.
DNS and TLS	Tenant wildcard <code>*.roserp.workvar.com</code> , certificates terminated at the proxy.
Analytics	Google Analytics 4 and Microsoft Clarity, both no-ops when the IDs are unset.
Marketing and legal	<code>peepal.app</code> , with <code>/terms</code> and <code>/privacy</code> .
On-premises	The <code>installer/</code> module builds native installers plus a <code>peepal-agent</code> supervisor that proxies port 80, serves a maintenance page, and auto-updates from private release repositories.

15.2 Environment variables

Frontend (`.env.local` or the PM2 env block):

Variable	Purpose
<code>NEXT_PUBLIC_API_URL</code>	Origin of the Go API
<code>NEXT_PUBLIC_GA_MEASUREMENT_ID</code>	GA4 measurement ID; empty locally and in CI
<code>NEXT_PUBLIC_CLARITY_PROJECT_ID</code>	Optional Clarity project ID
<code>NODE_ENV</code> , <code>PORT</code>	Production process settings

Backend (see `.env.example` for the full list):

Variable	Purpose
<code>DB_PATH</code>	PostgreSQL DSN
<code>PORT</code>	API listen port; defaults to 8080 when unset
<code>COOKIE_DOMAIN</code>	Cookie scope. Must be empty on localhost.
JWT and admin seed secrets	Unique per environment; rotate on compromise
SMTP settings	Invitation and notification delivery

Never commit a real `.env` or `.env.local`. Use a secret manager or host-level injection in production.

15.3 Networking checklist

1. Allowlist the application hosts on the database, or use private networking.
2. Expose only 443 publicly, with 80 redirecting. Keep the app ports on localhost or a private interface.
3. Configure CORS and cookie domains consistently with the tenant hostnames and any custom domains.

4. Ensure the reverse proxy forwards cookies and does not strip the tenant header.

15.4 Observability

Signal	Where
Process health	<code>pm2 status</code> , <code>pm2 logs</code>
Product analytics	GA4; the event registry is listed at <code>/dev/analytics</code>
Session replay	Microsoft Clarity
Platform health	<code>/super/dashboard</code> , live from the <code>platformAnalytics</code> query
Database	Aiven console: connections, disk, backups
Change history	The in-app Audit Trail, per tenant

16. Security, backups and support

Topic	Practice
Secrets	Unique JWT and admin secrets per environment; rotate after staff offboarding.
Passwords	bcrypt hashed. Sessions are 24-hour JWTs in httpOnly cookies.
Authorisation	Enforced on the API through resolver guards and access field middleware. Frontend guards are UX only.
Encryption	At rest and in transit on the managed database. Sensitive fields such as payment details carry column-level encryption on top.
Backups	Managed database schedules, plus the uploads volume. Test a restore quarterly.
Least privilege	Prefer role templates over per-user grants; audit administrator accounts monthly.
Production access	Engineering access to production data is exceptional, ticketed and logged.
Data residency	Confirm region placement with the platform owner for DPDP, GDPR or local compliance.
Incidents	Capture <code>pm2 logs</code> , the approximate UTC time, the tenant slug and the user role before escalating.

Support escalation: institute administrator, then Peepal support, then engineering on call.

17. Quick reference

Item	Value
Frontend dev	<code>pnpm dev</code> at <code>http://localhost:3000</code>
Backend dev	<code>go run main.go</code> , port from <code>PORT</code> (3001 in <code>.env.example</code> , 8080 default)
Apply schema	<code>go run main.go --migrate</code>
GraphQL endpoint	<code>POST {API}/api/v1/graphql</code>
Production API	<code>https://api-roserp.workvar.com</code>
Auth cookie	<code>peepal_token</code> , <code>httpOnly</code> , 24 hours
REST envelope	<code>{ success, data, message, error }</code>
Database	Aiven PostgreSQL via <code>DB_PATH</code>
Process manager	PM2 with <code>ecosystem.config.js</code>
Uploads	<code>backend/uploads/</code> on persistent disk
Package manager	pnpm for the frontend
Toolchain	Node 18+ (20 recommended), Go 1.21+ (project targets 1.25)
Default seed admin	<code>admin@college.edu</code> / <code>Admin@123</code>
In-app docs	<code>{tenant}/docs/dev</code> , <code>{tenant}/docs/users</code> , <code>/super/tech-docs</code>
Schema visualiser	<code>/super/db-visualizer</code>
Analytics registry	<code>/dev/analytics</code>
Legal	<code>peepal.app/terms</code> , <code>peepal.app/privacy</code>

A. Glossary

Term	Meaning
Tenant	One institution with isolated data, reached by subdomain or path slug.
Industry / tenant type	education, healthcare, corporate or nonprofit. Decides which modules exist. <code>college</code> and <code>enterprise</code> are legacy aliases for education and corporate.
Module	A named feature area that can be granted, gated by subscription, and filtered by industry.
Access matrix	The per-tenant grid of role to module CRUD permissions, edited at <code>/org/access-control</code> .
Role ceiling	The maximum a custom role can reach, derived from the resolver role guards of each module.
Role-locked module	A page whose data only exists for one role, hidden even from administrators.
Terminology layer	Per-tenant vocabulary tokens that rename entities in the interface without changing the data model.
Quota	The licensed student and employee count. Exceeding it blocks new records and shows a persistent banner.
Super admin	Platform operator outside any tenant, working from the <code>/super</code> console.
Bulk framework	The shared CSV import system: a registry, one schema per resource, and a reusable upload button.
UHID / CR number	Healthcare identifiers for a patient and an encounter respectively.

B. Industry availability matrix

A condensed view of which module families exist for which industry. Shared modules appear for every tenant.

Module family	Education	Healthcare	Corporate / Nonprofit
Dashboard, Profile, Ask PeepalAI	Yes	Yes	Yes
Employees, Org structure, Departments	Yes	Yes	Yes
Attendance (core)	Yes	Yes	Yes
Attendance summary, shortage, export	Yes	No	No
Students, Marks, Results, Courses, Subjects	Yes	No	No
Curriculum, Grading, Timetable, Exams	Yes	No	No
Exam cell (question bank, papers, hall tickets)	Yes	No	No
Fees (all sub-modules)	Yes	No	No
Hostel, Transport, Library, Mess	Yes	No	No
Student portal (My Fees, My Grades, ...)	Yes	No	No
Patients, Appointments, OPD, Admissions	No	Yes	No
Laboratory, Radiology, Nursing, OT, Triage	No	Yes	No
Pharmacy, Inventory, Blood bank, Ambulance	No	Yes	No
Clinical billing, Claims	No	Yes	No
Dietary, Telemedicine, Referrals	No	Yes	No
Wards & Beds, Clinician schedules, Patient portal	No	Yes	No
Leaves, Payroll, Approvals, Duty roster	Yes	Yes	Yes
Finance, Vendors, Purchase orders	Yes	Yes	Yes
Events, Learning, Announcements, Notifications	Yes	Yes	Yes
Organisation settings, Access control, Audit trail	Yes	Yes	Yes

Availability is also subject to your subscription. The authoritative source is the live access matrix at </org/access-control> and the module configuration in the super-admin console, not this table.

Peepal ERP Onboarding Guide, version 1.0, August 2026. Derived from the project conventions, the backend and frontend documentation, and the live module registry. Update this document when ports, domains, roles or migration policy change.